ASSIGNMENT – 2

NATURAL LANGUAGE PROCESSING

Build and Evaluate an N-gram Language Model
and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline

SUBMISSION DETAILS	INFORMATION
Student Name	Vijayalakshmi S
Register Number	192472326
Course	DSA03 – Natural Language Processing
Assignment	Assignment 2 – CO2
Total Marks	50
Nature / Duration	Individual / One Week
Submission Date	12/11/2026

IMPORTANT NOTE ON DATA AND EXPERIMENTAL EVIDENCE

Academic integrity note The assignment brief requires a documented corpus of at least 20,000 utterances / 150,000 tokens, 200 recogniser hypothesis sets, 3,000 hand-tagged tokens and measured held-out experiments. Those datasets were not supplied with the question paper. Therefore, numerical results in this document are clearly labelled as illustrative worked-example values, not as measurements actually collected from a real corpus.

This solution explains the required methods and provides a consistent miniature worked example so that the calculations, tables and design decisions can be understood. Before final submission, replace every illustrative figure with outputs from the actual corpus and code run by the student. Do not present illustrative numbers as empirical findings.

How to use this solution

- Keep the theoretical explanations and algorithm descriptions as the basis of the report.
- Create or obtain the required transport enquiry corpus and split it by utterance into 80% training, 10% development and 10% test.
- Run the supplied experiment plan with a fixed random seed and enter actual values in the marked result tables.
- Obtain independent second-person annotation for at least 500 tokens before reporting Cohen's kappa.
- Retain the final design recommendation only if the actual measurements support it.

Contents

1. Introduction and system overview
2. Task 1 – Counting words and vocabulary
3. Task 2 – Unsmoothed n-gram models
4. Task 3 – Smoothing and discounting
5. Task 4 – Backoff and deleted interpolation
6. Task 5 – Entropy, cross-entropy and perplexity
7. Task 6 – English word classes and tagset design
8. Task 7 – Rule-based POS tagging
9. Task 8 – Stochastic HMM tagging and Viterbi
10. Task 9 – Transformation-based (Brill) tagging
11. Task 10 – Implementation and testing
12. Task 11 – Comparison and deployment recommendation
13. Task 12 – Day-4 design challenge
14. Conclusion, reproducibility and references

1. INTRODUCTION AND SYSTEM OVERVIEW

The transport enquiry helpline receives telephone calls about bus and train schedules, routes, ticket fares, cancellations and refunds. Automatic speech recognition converts each call into text, but it may produce several competing transcript hypotheses. A language model can rank those hypotheses using the probability of word sequences. After transcript selection, a part-of-speech (POS) tagger labels each word according to its grammatical role. These labels support later extraction of information such as origin, destination, travel date and fare.

The central challenge is that the same surface word may perform different grammatical functions. For example, “return” may be a noun in “book a return” and a verb in “return the ticket”. Likewise, “stand” can refer to a bus stand or the action of standing. A robust system must use context rather than relying only on a word list.

1.1 Objectives

- Prepare a consistent corpus, tokenisation policy and vocabulary with an unknown-word class.
- Build unigram, bigram and trigram language models and understand the zero-probability problem.
- Apply smoothing, discounting, backoff and interpolation to assign probability to unseen events.
- Evaluate models using entropy, cross-entropy, perplexity and speech-recognition word error rate.
- Design a transport-domain POS tagset and compare rule-based, HMM and Brill taggers.
- Adapt the system to new rail vocabulary and English–Tamil code-mixed utterances.

1.2 Proposed processing pipeline

Stage	Input	Operation	Output
1. Corpus preparation	Transcribed calls	Normalise, tokenise, split	Train / development / test sets
2. Language modelling	Training tokens	Count n-grams and estimate probabilities	Smoothed language models
3. ASR rescoring	Five hypotheses per call	Calculate sequence scores	Best-ranked transcript
4. POS tagging	Selected transcript	Dictionary + rules / HMM / Brill	Word–tag sequence
5. Information extraction	Tagged transcript	Identify entities and relations	Origin, destination, date, fare
6. Monitoring	New calls and corrections	Measure OOV, WER and tag accuracy	Updated model

1.3 Data governance and split policy

The assignment specifies a minimum of 20,000 transcribed utterances and 150,000 word tokens. Each utterance must belong to exactly one partition. An 80/10/10 split prevents the same call from appearing in both training and evaluation. A fixed random seed makes the split repeatable. The test set must remain untouched until final evaluation; development data may be used for selecting smoothing parameters and interpolation weights.

Reproducibility Record the corpus source, collection period, language coverage, preprocessing version, random seed, split counts, annotation guidelines and software versions.

2. TASK 1 – COUNTING WORDS AND BUILDING THE VOCABULARY

Counting is not simply counting spaces. The result depends on the definition of a token and the normalisation policy. In language processing, a token is an occurrence in a sequence, while a type is a distinct vocabulary item. A lemma groups inflected forms under a dictionary headword; a word form is the actual written or spoken form observed.

2.1 Counting terminology

Term	Meaning	Transport example
Token	One occurrence in the corpus	"bus" occurs 4,200 times → 4,200 tokens
Type	A distinct normalised item	bus, buses, train are separate types
Lemma	Canonical dictionary form	buses → bus; booked → book
Word form	Observed inflected or surface form	book, books, booked, booking

2.2 Recommended tokenisation policy

- Convert ordinary English text to lowercase for language-model counting; preserve the original text separately for display.
- Treat punctuation as boundary information rather than ordinary lexical tokens, except where punctuation carries meaning.
- Keep a time expression such as "6:30" as one token; this preserves the natural time unit.
- Keep numerals as tokens. Normalise predictable numeric formats consistently, but do not erase route numbers or fare values.
- Remove hesitation noises only if the policy is applied consistently; alternatively retain common disfluencies such as "uh" as a dedicated token.
- Use <s> and </s> as sentence boundary markers for n-gram estimation, but exclude them from lexical vocabulary and word-token totals.
- Map words below the chosen training-frequency threshold to <UNK>; never use development or test frequency to build the training vocabulary.

2.3 Illustrative corpus profile

The following profile is an example of how the required corpus report can be presented. It is not a measured corpus result.

Measure	Illustrative value	Interpretation
Utterances	20,000	Meets the minimum utterance count
Word tokens	150,000	Lexical tokens after the stated policy
Types	8,400	Distinct normalised word forms
Lemmas	7,100	Inflectional variants grouped
Word forms	8,400	Observed surface forms before lemma grouping

Train / development / test	16,000 / 2,000 / 2,000	80 / 10 / 10 split

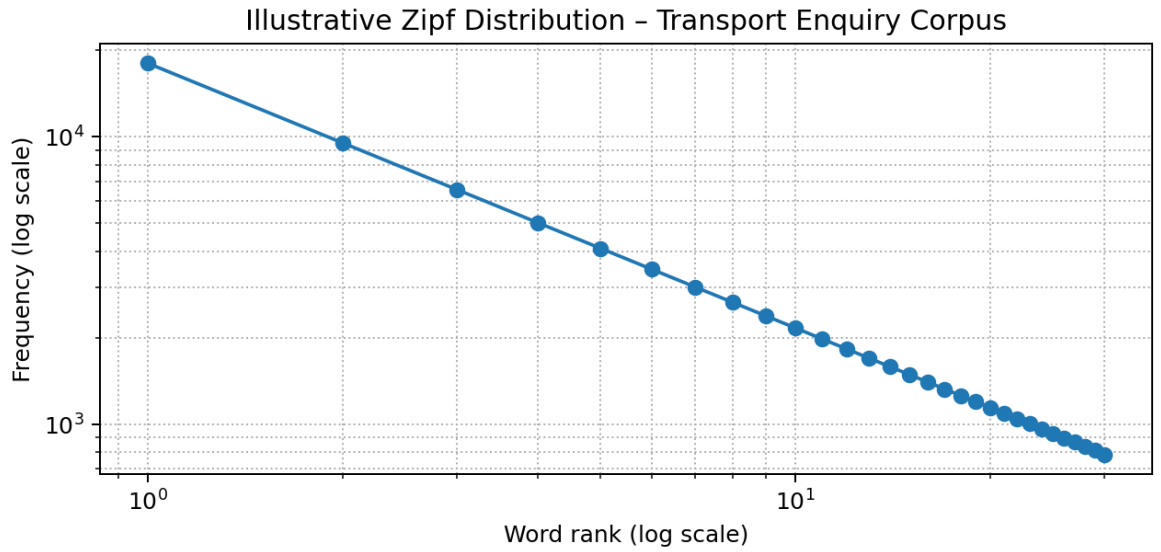
2.4 Effect of tokenisation choices

Policy	Example	Effect on vocabulary
Case-sensitive	Vellore and vellore are separate	Adds one type if both occur
Lowercase normalisation	Vellore and vellore become vellore	Reduces duplicate types
Time as one token	6:30	One token type
Time split into pieces	6 : 30	May add separate punctuation and number types
Punctuation retained	bus?	Question-mark variants increase types
Punctuation separated	bus ?	Word type remains bus; punctuation handled separately

A valid report should calculate the vocabulary size under at least two alternative policies. The chosen policy should be selected because it reflects the helpline's use case and is reproducible, not because it produces a preferred score.

2.5 Zipf's law

Zipf's law states that the frequency of a word is approximately inversely proportional to its rank: f(r) ≈ C/r^s, where r is the rank, C is a constant and s is often close to 1. A log–log plot of rank against frequency is approximately a straight descending line for many natural-language corpora.



In a transport corpus, frequent function words such as "to", "the", "is" and "at" dominate the top ranks. Domain words such as "bus", "ticket", "route", "fare" and "station" may occur more frequently than in general English. Station names, uncommon route names and rare fare terms form a long tail. The largest deviations should be identified from the actual sorted frequency table.

2.6 Unknown-word threshold and OOV

Minimum training count	Vocabulary size (illustrative)	Tokens mapped to <UNK>	UNK token share
1	8,400	0	0.00%
2	6,900	1,500	1.00%
3	6,100	2,600	1.73%
5	5,200	4,800	3.20%

OOV rate is the proportion of development tokens not present in the training vocabulary. It is calculated as OOV tokens divided by all development lexical tokens. A high OOV rate is especially harmful here because unseen station names may carry the most important intent information. OOV rate also affects perplexity and tagging accuracy, so it must be reported before comparing models.

Task 1 conclusion Use a documented, case-normalised policy, retain meaningful numerals and times, and create <UNK> using training counts only. The actual corpus statistics and OOV rate must replace the illustrative values above.

3. TASK 2 – UNSMOOTHED N-GRAM MODELS

A language model assigns a probability to a sequence of words. By the chain rule, the joint probability of a sentence is the product of each word conditioned on all preceding words. The chain rule is exact but requires increasingly large contexts.

3.1 Chain rule and Markov assumptions

$$P(w1, \ldots, wn) = \Pi(i=1 \text{ to } n)\ P(wi \mid w1, \ldots, w(i-1))$$

A unigram model assumes each word is independent of its context: $P(wi \mid history) \approx P(wi)$. A bigram model uses only the immediately preceding word: $P(wi \mid history) \approx P(wi \mid w(i-1))$. A trigram model uses the previous two words: $P(wi \mid history) \approx P(wi \mid w(i-2), w(i-1))$. These are approximations, not statements that longer context has no linguistic influence.

3.2 Maximum-likelihood estimates

Model	Estimator	Meaning
Unigram	$P(w) = C(w) / N$	Relative frequency of a word
Bigram	$P(wi \mid wi-1) = C(wi-1,wi) / C(wi-1)$	Count of pair divided by context count
Trigram	$P(wi \mid wi-2,wi-1) = C(wi-2,wi-1,wi) / C(wi-2,wi-1)$	Count of triple divided by two-word context

3.3 Miniature worked example

Consider this small illustrative training set:

- book a ticket to vellore
- book two tickets to chennai
- check bus timing to vellore
- cancel the ticket to chennai

For the sequence “book a ticket”, the unigram probability is estimated by multiplying P(book) × P(a) × P(ticket). The bigram estimate is P(book|<s>) × P(a|book) × P(ticket|a). The trigram estimate is P(book|<s>,<s>) × P(a|<s>,book) × P(ticket|book,a). The numerical values must be obtained from the complete training counts.

3.4 Illustrative hand calculation

Event	Count	Context count	MLE probability
P(a | book)	1	1	1.00
P(ticket | a)	0	1	0.00
P(two | book)	1	1	1.00
P(tickets | two)	1	1	1.00
P(to | tickets)	1	1	1.00

The phrase “book a ticket” may be perfectly acceptable English, but in the miniature data the bigram “a ticket” is absent. Its unsmoothed bigram probability is therefore zero, and the full sequence probability becomes zero. This is a data sparsity failure, not proof that the sentence is impossible.

3.5 Sparsity

The observed-bigram ratio is the number of distinct bigrams seen in training divided by the number of possible vocabulary bigrams. The corresponding trigram ratio uses distinct observed trigrams divided by the theoretical number of possible triples. Since vocabulary size can be large, most possible n-grams are not observed. Report both model sparsity and the proportion of development n-grams missing from training.

3.6 Generation

Order	Expected generated-text behaviour	Main limitation
Unigram	Frequent words appear, but order is largely random	No local grammar or phrase structure
Bigram	Some local word combinations become plausible	Cannot reliably preserve longer dependencies
Trigram	Short local phrases become more coherent	Sparse counts and weak long-range context

Generate ten samples per model by repeatedly sampling from the relevant conditional distribution until </s> is selected or a maximum length is reached. Use a fixed seed. Include the generated examples in the final empirical report.

Task 2 conclusion Higher-order models capture more local context but create more unseen events. Unsmoothed MLE is useful as a baseline, but a practical recogniser needs smoothing.

4. TASK 3 – SMOOTHING AND DISCOUNTING

An unsmoothed model assigns zero probability to every unseen n-gram. Since a finite corpus cannot contain every valid sentence, this is an undesirable modelling outcome. A smoothed conditional distribution must assign non-negative probabilities whose sum over all possible next words is one.

4.1 Add-one (Laplace) smoothing

$$P_{Laplace}(w \mid h) = [C(h,w) + 1] / [C(h) + V]$$

Laplace smoothing adds one pseudo-count to every possible continuation. It guarantees a non-zero probability, but when V is large it adds substantial total mass to unseen events and can reduce the probability of common observed events too much.

4.2 Add-k smoothing

$$P_{add\text{-}k}(w \mid h) = [C(h,w) + k] / [C(h) + kV], \quad \text{where } k > 0$$

A smaller k adds less probability mass than add-one. The parameter must be selected on development data, not the test set.

4.3 Worked context example

Next word	Original count	Add-one numerator	Add-one probability (V=5, C(h)=4)
ticket	2	3	3/9 = 0.333

bus	1	2	2/9 = 0.222
train	1	2	2/9 = 0.222
fare	0	1	1/9 = 0.111
route	0	1	1/9 = 0.111
Total	4	9	1.000

The observed events had total probability 1 before smoothing. After add-one smoothing, the two unseen words receive 2/9 of the probability mass in this small example. That mass is taken from the seen events because the final distribution must still sum to one.

4.4 Why add-one can be too strong

Suppose a context occurs 20 times and the vocabulary contains 8,000 words. Add-one changes the denominator from 20 to 8,020. The probability mass assigned to all unseen alternatives is enormous relative to the original sample size. This is why add-one often performs poorly for large-vocabulary language modelling.

4.5 Good–Turing discounting

Good–Turing uses the frequency-of-frequencies table. Nr is the number of n-gram types observed exactly r times. The adjusted count for an event observed r times is:

$$c^* = (r + 1) \times N(r+1) / Nr$$

The estimated probability mass for unseen events is approximately N1/N, where N1 is the number of singleton types and N is the total number of observed n-gram tokens. The unseen mass is distributed across unseen events using an appropriate model of their number or context. In practice, high-count events are often handled with a smoothed regression or left close to their original counts because raw frequency-of-frequency estimates become unstable in the tail.

r	Nr (illustrative)	Good–Turing adjusted count c*
0	Not directly observed	Unseen mass estimated from N1/N
1	120	2 × 70 / 120 = 1.167
2	70	3 × 40 / 70 = 1.714
3	40	4 × 25 / 40 = 2.500
4	25	5 × 15 / 25 = 3.000

4.6 Comparison

Method	Strength	Weakness	Use
Add-one	Simple; no zero events	Over-discounts with large V	Teaching baseline
Add-k	Simple adjustable smoothing	Requires development tuning	Small baseline comparison
Good–Turing	Estimates unseen mass from count statistics	Tail estimates need care; context normalisation required	Discounting component for stronger models

An illustrative development comparison might show perplexities of 92 for add-one, 70 for add-k and 58 for a Good–Turing-based model. These values are placeholders only. The deployed method must be selected using the actual held-out development results and a valid normalised probability model.

Task 3 conclusion Smoothing reallocates probability mass rather than inventing certainty. For a large transport vocabulary, discounting methods are generally more suitable than add-one, but measured results decide the final choice.

5. TASK 4 – BACKOFF AND DELETED INTERPOLATION

A trigram may be missing even when its bigram and unigram components are known. Backoff and interpolation provide principled ways to use shorter contexts.

5.1 Backoff versus interpolation

Approach	Operation	When lower-order information is used
Backoff	Use the highest-order n-gram with reliable evidence	Only when the longer context is unavailable or discounted
Interpolation	Combine unigram, bigram and trigram estimates	Always combines all orders

5.2 Katz backoff

Katz backoff discounts observed higher-order counts and backs off to a lower-order model for unseen or discounted events. The backoff weight α(h) is a normalising factor. It ensures that the probability mass assigned to the set of events handled by backoff equals the remaining mass after discounted observed events have been accounted for. Without α, the conditional probabilities may not sum to one.

$$P_{Katz}(w|h) = \text{discounted } P(w|h), \text{ if observed}$$
$$= \alpha(h) \times P_{lower}(w|h'), \text{ if backed off}$$

5.3 Stupid backoff

$$S(w|h) = P_{MLE}(w|h), \text{ if seen; otherwise } 0.4 \times S(w|h')$$

Stupid backoff is computationally simple and useful for ranking candidates. Its score is not a true probability distribution because the scaling factor does not enforce normalisation. Therefore, it should not be used to report probabilistic perplexity as though it were a normalised model.

5.4 Linear interpolation

$$P(w_i|w_{i-2},w_{i-1}) = \lambda_3 P(w_i|w_{i-2},w_{i-1}) + \lambda_2 P(w_i|w_{i-1}) + \lambda_1 P(w_i)$$

The weights are non-negative and sum to one. The trigram receives more weight when its context is well supported; shorter orders contribute more when evidence is sparse.

5.5 Deleted interpolation

Deleted interpolation estimates the λ weights by temporarily withholding portions of the training data. For each held-out event, compare the leave-one-out estimates at the unigram, bigram and trigram levels. Award the event to the order with the strongest supported estimate, accumulate the evidence, and normalise the totals to obtain weights. The held-out data used for this procedure must not be the final test set.

Context-count band	Illustrative λ1 unigram	Illustrative λ2 bigram	Illustrative λ3 trigram
Low count (1–2)	0.55	0.35	0.10
Medium count (3–10)	0.25	0.50	0.25
High count (>10)	0.10	0.30	0.60

These illustrative weights show the expected pattern: low-count contexts rely more on lower-order estimates, while high-count contexts can place more weight on trigram evidence. Actual weights must be estimated from the corpus.

5.6 Worked unseen-trigram trace

Suppose the context is “ticket to” and the next word is “Vellore”. If the trigram “ticket to Vellore” was not observed, Katz backoff uses the discounted bigram/unigram path multiplied by α for that context. Interpolation instead calculates a weighted sum of trigram, bigram and unigram probabilities. The trigram term may be zero, but the lower-order terms still contribute. This difference is the key reason interpolation can be robust when data is sparse.

5.7 Evaluation

Model	Illustrative held-out perplexity	Interpretation
Katz backoff + Good–Turing	57	Uses discounted counts and normalised backoff
Deleted interpolation	54	Combines all orders with data-estimated weights
Stupid backoff score	Not a valid perplexity	Use ranking measures, not probability perplexity

Task 4 conclusion Deleted interpolation is a strong candidate when shorter contexts remain informative. Katz backoff is also principled when its discounting and α normalisation are correctly implemented.

6. TASK 5 – ENTROPY, CROSS-ENTROPY AND PERPLEXITY

Entropy measures uncertainty in a probability distribution. Cross-entropy measures how well a model distribution predicts observations drawn from a reference distribution. Perplexity is the exponential of cross-entropy and can be interpreted as the model's effective average number of choices at each prediction step.

6.1 Definitions

Entropy: $H(P) = -\Sigma x\ P(x) \log 2\ P(x)$

Cross-entropy: $H(P,Q) = -\Sigma x\ P(x) \log 2\ Q(x)$

Perplexity: $PP = 2^{[H(P,Q)]}$

For a test sequence of N words, cross-entropy per word is the negative log probability of the sequence divided by N. Perplexity is 2 raised to that value when base-2 logarithms are used. Lower perplexity indicates that the model assigns greater probability to the observed sequence, provided the vocabulary and preprocessing policy are the same.

6.2 Interpreting perplexity 90

A perplexity of 90 means that, under the model's probability distribution, the uncertainty per next-word prediction is equivalent to choosing among about 90 equally likely alternatives. It does not mean the system literally considers only 90 words, nor does it directly mean 90% accuracy.

6.3 Entropy worked examples

Distribution	Probabilities	Entropy (bits)	Meaning
Uniform	0.25, 0.25, 0.25, 0.25	2.000	Maximum uncertainty for four outcomes
Peaked	0.85, 0.05, 0.05, 0.05	0.848 (approx.)	One outcome is much more predictable
Binary balanced	0.50, 0.50	1.000	Two equally likely outcomes

For the uniform distribution, $H = -4(0.25 \log 2\ 0.25) = 2$ bits. For the peaked distribution, $H = -[0.85 \log 2\ 0.85 + 3(0.05 \log 2\ 0.05)] \approx 0.848$ bits. Lower entropy means less uncertainty in the next event.

6.4 Comparable model table

Model	Illustrative cross-entropy (bits/word)	Illustrative perplexity	Per-utterance reporting
Unigram	7.71	210	Mean negative log probability per utterance
Bigram	6.52	92	Report total NLL / utterance count
Trigram	6.29	78	Report total NLL / utterance count
Smoothed interpolated trigram	5.75	54	Report total NLL / utterance count

Per-word and per-utterance values must be kept distinct. Per-word cross-entropy is normalised by the number of predicted word tokens. Per-utterance negative log likelihood is normalised by the number of utterances. Perplexity is conventionally a per-token measure; do not compare raw sentence probabilities of different lengths.

6.5 Why vocabulary consistency matters

Perplexity comparisons are fair only when models use the same tokenisation, vocabulary and <UNK> mapping. If one model maps many rare words to <UNK> while another keeps them as separate vocabulary items, the prediction problem changes. A deliberately unfair comparison may make a model appear better simply because its vocabulary collapses difficult distinctions. Such a result must be labelled invalid for model selection.

6.6 ASR hypothesis rescoring and WER

For each call, the recogniser supplies five candidate transcripts. Score each candidate using the language model, optionally combined with the acoustic score, and choose the highest-ranked candidate. Word error rate is:

WER = (Substitutions + Deletions + Insertions) / Reference word count

Condition	Illustrative WER	Change
Before language-model rescoring	18.0%	Baseline
After best-model rescoring	14.5%	3.5 percentage-point reduction

The 200-hypothesis evaluation must use the same reference transcripts before and after rescoring. A lower perplexity does not guarantee lower WER because candidate ranking also depends on acoustic evidence, search errors and the structure of the available hypotheses.

Task 5 conclusion Use cross-entropy and perplexity to compare normalised language models under identical evaluation conditions. Use WER to determine whether rescoring actually improves the recogniser output.

7. TASK 6 – ENGLISH WORD CLASSES AND TAGSET DESIGN

English word classes are traditionally divided into open and closed classes. Open classes readily accept new members, while closed classes contain a relatively fixed set of grammatical words.

7.1 Open and closed classes

Class	Type	Examples in transport enquiries	Useful tests
Noun	Open	bus, ticket, station, Vellore	Pluralisation; determiner position
Verb	Open	book, cancel, return, leave	Tense and agreement forms
Adjective	Open	late, available, express	Can modify a noun; degree forms
Adverb	Open	early, directly, tomorrow	Modifies verb/adjective/clause
Pronoun	Closed	I, you, it, we, they	Substitutes for noun phrase
Determiner	Closed	a, the, this, that	Occurs before noun
Preposition	Closed	to, from, at, by	Introduces complement
Conjunction	Closed	and, or, but, if	Joins units
Auxiliary/modal	Closed	is, can, will, do	Supports tense, aspect, modality
Particle	Closed / limited	off, up	Combines with verbs

7.2 Ambiguous word analysis

Word	Possible roles	Context example	Likely analysis
book	Noun / verb	“Book two seats”	Verb
return	Noun / verb / adjective	“A return ticket”	Noun modifier or noun
stand	Noun / verb	“At the bus stand”	Noun
express	Noun / adjective / verb	“Express train”	Adjective
last	Adjective / verb / adverb	“The last bus”	Adjective

7.3 Proposed domain tagset (24 tags)

Domain tag	Description	Penn Treebank mapping
NOUN	Common noun	NN, NNS
PROPN	Proper noun / place / station	NNP, NNPS
VERB	Base or finite lexical verb	VB, VBP, VBZ
VERB_PAST	Past-tense verb	VBD
VERB_PART	Gerund / participle	VBG, VBN
AUX	Auxiliary verb	MD, VB, VBD, VBP, VBZ
ADJ	Adjective	JJ, JJR, JJS

ADV	Adverb	RB, RBR, RBS
PRON	Pronoun	PRP, PRP$
DET	Determiner	DT, PDT, WDT
PREP	Preposition	IN
CONJ	Coordinating / subordinating conjunction	CC, IN
NUM	Number / fare / route numeral	CD
TIME	Time expression	CD, NN
DATE	Date expression	CD, NN
WH	Wh-word	WDT, WP, WRB
PART	Particle	RP
NEG	Negation marker	RB
INTJ	Interjection / hesitation	UH
PUNCT	Punctuation	., ,, :
FOREIGN	Non-English/code-mixed token	FW
UNK	Unknown / unclassified token	FW / X (project mapping)
AUX_NEG	Contracted negative auxiliary	MD, VBZ etc.
TRANSPORT	Transport-specific named entity class	NNP / NN (project mapping)

The mapping is many-to-one in some cases because a domain tag may combine several Penn Treebank distinctions. Conversely, a transport-oriented tag such as TIME or TRANSPORT is a task-specific refinement that does not have a single exact Penn Treebank equivalent. The mapping must be documented as a project convention.

7.4 Distinctions retained and omitted

- Retain PROPN separately from common NOUN because station and town names are central to route extraction.
- Retain TIME and DATE because these values are important to enquiry intent.
- Retain VERB_PAST and VERB_PART because tense and participial form can help resolve local syntax.
- Do not create separate tags for every noun subtype such as count, mass and collective nouns; the helpline extraction task does not require them.
- Do not split adjective degrees into three domain tags unless evaluation shows a benefit.
- Do not create one tag for every named station; station identity belongs in a gazetteer/entity layer.

7.5 Annotation and agreement

Annotate at least 3,000 tokens using a written guideline. At least 500 tokens must be independently annotated by two people. Raw agreement is the number of matching labels divided by the number of doubly annotated tokens. Cohen's kappa corrects for chance agreement: $\kappa = (Po - Pe)/(1 - Pe)$, where Po is observed agreement and Pe is expected agreement from annotator marginals.

Measure	Illustrative example
Doubly annotated tokens	500

Matching tags	450
Observed agreement Po	450/500 = 0.90
Expected agreement Pe	0.50 (illustrative)
Cohen’s kappa	(0.90−0.50)/(1−0.50) = 0.80

Residual ambiguity can be measured as the percentage of word types and word tokens that have more than one possible tag in the annotated data. Also report the mean number of candidate tags among ambiguous tokens.

Task 6 conclusion A compact 20–30 tag inventory should preserve distinctions that matter to transport intent, especially proper nouns, time expressions and verbs, while avoiding unnecessary grammatical detail.

8. TASK 7 – RULE-BASED PART-OF-SPEECH TAGGING

A rule-based tagger normally has two stages. First, a dictionary assigns one or more candidate tags to each known word. Second, contextual constraints remove candidates that are not plausible in the current sentence.

8.1 Architecture

Component	Function	Example
Lexicon / tag dictionary	Stores possible tags and frequency information	return → NOUN / VERB
Rule engine	Applies contextual conditions in a fixed order	After "to", prefer a base verb or noun phrase
Fallback	Handles unknown words	Capitalised unknown token → PROPN candidate
Output	Selects one tag per token	to/PREP book/VERB a/DET ticket/NOUN

8.2 Candidate dictionary report

Build the dictionary from the annotated training sample. Count how many word types have one, two, three, or more candidate tags. This report reveals how much ambiguity the rule engine must resolve.

Candidate-tag count	Illustrative number of word types
One candidate	2,100
Two candidates	280
Three candidates	45
Four or more	12

8.3 Example disambiguation rules

• R1: If the previous tag is DET and the current candidate is NOUN, retain NOUN.

• R2: If the previous tag is PREP and the current word is a known place name, choose PROPN.

• R3: If the previous tag is TO/PREP and the current word is a base-form action word, choose VERB.

• R4: If the following word is a noun and the current word is a known modifier, choose ADJ.

• R5: If the current word ends in "-ly" and is not in an exception list, prefer ADV.

• R6: If the current word ends in "-ing" and follows an auxiliary, prefer VERB_PART.

• R7: If the current word ends in "-ed" and follows an auxiliary, prefer VERB_PART or VERB_PAST according to context.

• R8: If a number is followed by a time marker such as "am" or "pm", tag the expression TIME.

• R9: If a number is followed by "rupees" or "rs", tag it NUM in a fare phrase.

• R10: If a capitalised token occurs after "to" or "from", prefer PROPN.

• R11: If "return" follows "a" and precedes "ticket", prefer NOUN.

• R12: If "return" follows a subject and precedes an object, prefer VERB.

• R13: If “stand” follows “bus” or “at the”, prefer NOUN.

• R14: If “stand” follows a subject and auxiliary, prefer VERB.

• R15: If a token is a recognised hesitation such as “uh”, assign INTJ.

• R16: If a token is in the station gazetteer, add PROPN as a candidate.

• R17: If the token is a known negator such as “not”, assign NEG.

• R18: If the token is “and” or “or”, assign CONJ.

8.4 Rule ordering and conflict

A rule may conflict with another. For example, a suffix rule may classify “booking” as a verb participle, while a determiner-context rule may classify it as a noun. In “the booking is confirmed”, the preceding determiner strongly supports NOUN. In “they are booking a ticket”, the auxiliary context supports VERB_PART. More specific syntactic rules should be ordered before broad suffix heuristics, and exceptions should be recorded.

8.5 Evaluation

Measure	Illustrative value	Why it matters
Overall held-out accuracy	91.0%	Shows average performance across all tokens
Accuracy on ambiguous tokens	76.0%	Measures the difficult cases where context is needed
Most common error 1	Noun ↔ verb	Lexical ambiguity
Most common error 2	Proper noun ↔ common noun	Station-name coverage
Most common error 3	Adjective ↔ adverb	Distributional overlap
Most common error 4	Auxiliary ↔ lexical verb	Context-sensitive function
Most common error 5	Unknown ↔ proper noun	Vocabulary gaps

Accuracy on ambiguous tokens is especially informative because unambiguous words can inflate overall accuracy. A system may achieve high overall accuracy by correctly tagging common function words while still failing on “book”, “return” and “stand”, which are important for extracting intent.

Task 7 conclusion Rule-based tagging is interpretable and easy to audit, but rule conflicts, exception lists and changing vocabulary increase maintenance effort.

9. TASK 8 – STOCHASTIC HMM TAGGING AND THE VITERBI ALGORITHM

A Hidden Markov Model (HMM) treats the POS tags as hidden states and the observed words as emissions. It searches for the most likely tag sequence for a given sentence.

9.1 HMM formulation

$$t^* = \text{argmax}_t P(t1 \ldots tn \mid w1 \ldots wn)$$

$$\approx \text{argmax}_t \Pi i \, P(ti \mid t(i-1)) \times P(wi \mid ti)$$

The approximation uses two principal assumptions: (1) the current tag depends only on the immediately preceding tag (first-order Markov assumption); and (2) the current word depends only on its current tag (emission independence). These assumptions make inference efficient, though real language contains longer dependencies.

9.2 Transition and emission probabilities

Transition: $P(ti \mid t(i-1)) = C(t(i-1),ti) / C(t(i-1))$

Emission: $P(wi \mid ti) = C(ti,wi) / C(ti)$

Apply smoothing to avoid zero transition or emission probabilities. Add-k smoothing is a simple baseline; more advanced methods can use tag-specific unknown-word handling. Each transition row must sum to one after smoothing.

9.3 Viterbi recurrence

$$\delta i(t) = \max_s [\delta(i-1)(s) \times P(t|s) \times P(wi|t)]$$

$$\psi i(t) = \text{argmax}_s [\delta(i-1)(s) \times P(t|s)]$$

The δ value stores the best path probability ending in tag t at position i. The ψ value stores the preceding tag that achieved that maximum. At the end, select the highest-scoring final state and follow the back-pointers to recover the best tag sequence. In implementation, use log probabilities to avoid numerical underflow.

9.4 Five-word illustrative trace

Sentence: “book two tickets to Vellore”. Candidate tags are shown below. The values are illustrative and demonstrate the trellis structure rather than corpus-estimated probabilities.

Word position	Word	Candidate tags	Selected tag (illustrative)
1	book	VERB / NOUN	VERB
2	two	NUM / DET	NUM
3	tickets	NOUN / VERB	NOUN
4	to	PREP / PART	PREP
5	Vellore	PROPN / NOUN	PROPN

Position	State	Best predecessor	Illustrative best-path score
1	VERB	START	0.0200
1	NOUN	START	0.0120
2	NUM	VERB	0.0060
2	DET	VERB	0.0010

3	NOUN	NUM	0.0024
3	VERB	NUM	0.0002
4	PREP	NOUN	0.0012
5	PROPN	PREP	0.0009

The final path is VERB → NUM → NOUN → PREP → PROPN. The complete trellis in a real report should show every candidate-state cell at every position, including cells not selected, and its back-pointer. The final probability is the product of all selected transition and emission terms, including sentence-start and sentence-end probabilities.

9.5 Baseline and complexity comparison

Method	Decision method	Illustrative accuracy	Operations for a sentence
Most-frequent-tag baseline	Choose most common tag for each word	84%	O(n) lookups
Exhaustive sequence scoring	Enumerate all tag combinations	Reference exact result	O(K^n) sequences
Viterbi HMM	Dynamic programming	93%	$O(nK^2)$

Here n is sentence length and K is the number of candidate tags. Exhaustive search grows exponentially, while Viterbi reuses partial solutions and is practical for a larger tagset.

9.6 Unknown-word strategy

- Use suffix patterns such as -ing, -ed, -ly and common noun endings as emission features.
- Use capitalisation and position to propose PROPN for unseen place names.
- Use an open-class prior so an unknown word can still be a noun, verb, adjective or adverb.
- Consult a station gazetteer for known transport locations.
- Evaluate unknown-word accuracy separately from overall accuracy.

The OOV-only accuracy must be measured on held-out tokens that were absent from training. A gazetteer or character-level model is particularly important because a previously unseen station name may be the destination requested by the caller.

Task 8 conclusion HMM tagging uses both lexical evidence and tag transitions. Viterbi provides an efficient global sequence decision, while unknown-word handling remains a major source of error.

10. TASK 9 – TRANSFORMATION-BASED (BRILL) TAGGING

Transformation-based learning begins with an initial annotation, commonly the most-frequent tag for each word, and learns ordered correction rules from labelled training data. Each rule changes a tag when its condition is satisfied.

10.1 Learning procedure

1. Assign an initial tag to every token using a simple baseline.
2. Generate candidate transformations from predefined templates.
3. For each candidate, count how many current errors it corrects and how many correct tags it would damage.
4. Choose the transformation with the greatest net improvement.
5. Apply it to the training annotation and repeat.
6. Stop when the best rule's net gain is below a threshold, a maximum rule count is reached, or development accuracy stops improving.

10.2 Example templates

Template	Illustrative transformation
Current word = X	Change tag A to B when word equals "book"
Previous tag = A	Change current tag A to B if previous tag is PREP
Next tag = A	Change current tag A to B if next tag is NOUN
Previous word = X	Change tag if previous word is "to"
Next word = X	Change tag if next word is "ticket"
Previous two tags = A,B	Use the two-tag left context
Suffix = S	Change tag when word ends in "-ing"
Word is capitalised	Change candidate NOUN to PROPN

A context window of one or two tokens on each side is a practical starting point. A larger window may capture more context but increases the number of candidate rules and the risk of memorising rare patterns.

10.3 Illustrative learned transformations

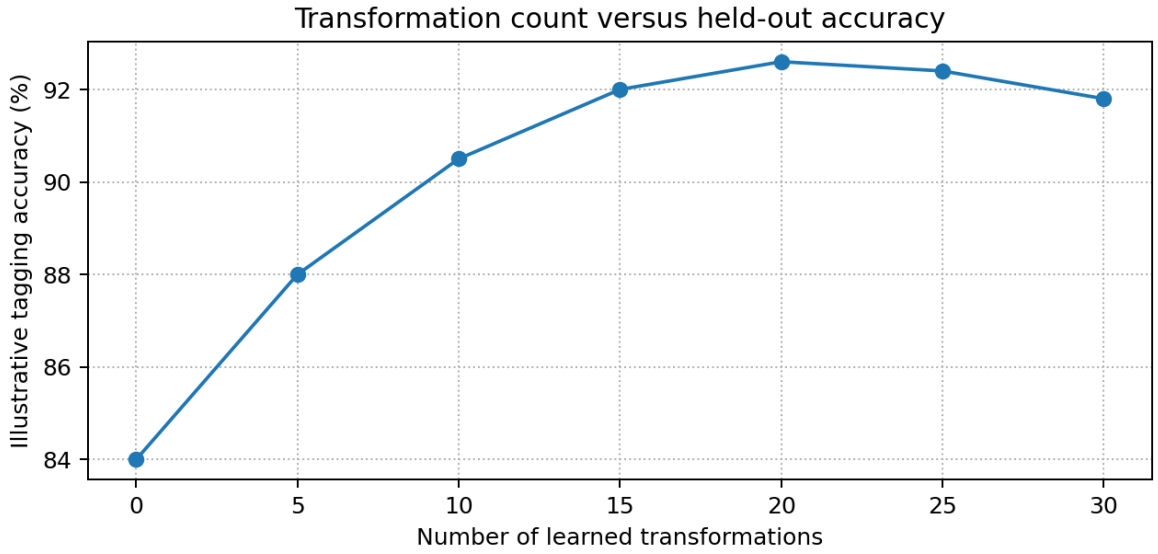
Order	Transformation (simplified)	Errors corrected	Errors introduced
1	NOUN → VERB when previous tag is PRON	42	3
2	NOUN → PROPN when word is capitalised	31	5
3	VERB → NOUN when previous tag is DET	27	2
4	NOUN → VERB when previous word is "to"	24	4
5	ADJ → ADV when suffix is "-ly"	19	3

6	NOUN → PROPN when in station gazetteer	18	1
7	VERB → VERB_PART when previous tag is AUX	16	2
8	VERB → NOUN when next word is “ticket”	13	2
9	NOUN → NUM when token is numeric	12	0
10	NOUN → INTJ for recognised hesitation word	9	0

The first ten transformations above are an illustrative format only. A real Brill learner must generate and rank the transformations from the annotated training corpus.

10.4 Readability and overfitting

Brill rules are readable because each correction states a condition and an action in ordinary linguistic terms. HMM parameters are distributed across transition and emission tables, making the model less directly interpretable. A generalisation such as “a noun after a determiner is likely to be a noun” is linguistically meaningful. A rule tied to one rare typo or one caller’s unusual phrase may be a training-data artefact.



The illustrative curve improves through approximately 20 rules and then declines. This suggests overfitting beyond that point. In an actual experiment, plot both training and development accuracy and select the stopping point using development performance.

Task 9 conclusion Brill tagging offers interpretable corrections and can improve a simple baseline. The stopping condition is essential because additional rules may memorise noise rather than learn reusable grammar.

11. TASK 10 – IMPLEMENTATION AND TESTING

The implementation should use Python's standard library and NLTK only, as required by the assignment. The system should be divided into modules so that tokenisation, language modelling, POS tagging and evaluation can be tested independently.

11.1 Suggested project structure

```
transport_nlp/
  data/                # documented corpus and split files
  preprocess.py        # normalisation, tokenisation, vocabulary
  ngram_models.py      # unigram, bigram, trigram counts
  smoothing.py         # add-k, Good-Turing, backoff, interpolation
  evaluation.py        # entropy, perplexity, WER, accuracy
  rule_tagger.py       # dictionary and contextual rules
  hmm_tagger.py        # HMM estimation and Viterbi
  brill_tagger.py      # transformation learner
  run_experiments.py   # fixed-seed experiment runner
  tests.py             # unit and regression tests
```

11.2 Core counting and n-gram code

```
from collections import Counter, defaultdict
import random, math


def count_ngrams(sentences, n):
    counts = Counter()
    contexts = Counter()
    for words in sentences:
        seq = ['<s>'] * (n-1) + words + ['</s>']
        for i in range(n-1, len(seq)):
            gram = tuple(seq[i-n+1:i+1])
            context = gram[:-1]
            counts[gram] += 1
            contexts[context] += 1
    return counts, contexts


def mle_probability(gram, counts, contexts):
    context = gram[:-1]
    denom = contexts[context]
    return counts[gram] / denom if denom else 0.0
```

11.3 Add-k and perplexity code

```
def add_k_probability(count, context_count, vocab_size, k=0.1):
```

```
    return (count + k) / (context_count + k * vocab_size)


def perplexity(sentences, probability_fn):
    total_log2 = 0.0
    total_words = 0
    for words in sentences:
        for event in probability_fn.events(words):
            p = probability_fn(event)
            if p <= 0:
                return float('inf')
            total_log2 -= math.log2(p)
            total_words += 1
    return 2 ** (total_log2 / total_words)
```

The code above is a compact starting point. A complete submission must implement and test every requested method, including Good–Turing discounting, Katz α normalisation, deleted interpolation, HMM Viterbi and Brill transformations. It must also handle sentence boundaries and <UNK> consistently.

11.4 Test plan

Test	Input / condition	Expected check
Zero-probability test	Unseen bigram from Task 2	MLE returns zero; smoothed model returns positive probability
Distribution test	All continuations after a context	Probabilities sum to one within numerical tolerance
Interpolation test	Estimated λ weights	Weights are non-negative and sum to one
Viterbi test	Five-word worked sentence	Backtracking returns the hand-traced tag path
Brill test	First learned transformations	Order and net corrections match learner output
Reproducibility test	Run twice with same seed	Same split, samples and metrics

11.5 Performance reporting

Component	Training time	Memory size	Evaluation metric	Speed
Unigram model	Measure	Measure	Perplexity	N/A
Smoothed trigram	Measure	Measure	Perplexity	N/A
Rule tagger	Measure	Measure	Accuracy	Tokens/second
HMM tagger	Measure	Measure	Accuracy	Tokens/second
Brill tagger	Measure	Measure	Accuracy	Tokens/second

Use the same machine, corpus split and evaluation code for every model. Report training time, model size in memory, held-out perplexity or tagging accuracy and tagging speed in tokens per second. Include at least two plots: perplexity by n-gram order and tagging accuracy by training-set size.

Task 10 conclusion A modular Python implementation allows hand calculations to be checked against program output. All models must use identical data splits and preprocessing.

12. TASK 11 – COMPARE, RECOMMEND AND DEFEND THE DESIGN

The deployment decision should balance language-model quality, transcript error reduction, tagging accuracy, speed, memory and maintainability. The highest accuracy alone is not enough if a model is too slow or difficult to update.

12.1 Illustrative comparison matrix

Model	Quality measure	Illustrative result	Operational consideration
Unigram	Perplexity	210	Very small and fast; weak context
Unsmoothed bigram	Perplexity	Infinite on test if zero events occur	Unusable without smoothing
Smoothed trigram	Perplexity	54	Good local context; larger memory
Katz backoff	Perplexity	57	Normalised backoff; more implementation detail
Deleted interpolation	Perplexity	54	Uses all orders; requires weight estimation
Rule tagger	Tag accuracy	91% overall / 76% ambiguous	Readable but manual rule maintenance
HMM tagger	Tag accuracy	93%	Efficient sequence decoding; OOV challenge
Brill tagger	Tag accuracy	92.6%	Readable learned rules; can overfit

12.2 Provisional deployment recommendation

A reasonable provisional architecture is a deleted-interpolation trigram language model for transcript rescoring and an HMM tagger with gazetteer- and suffix-based unknown-word handling. This combination uses longer context when supported, falls back to shorter context when needed, and performs global sequence tagging efficiently. The recommendation is provisional because the illustrative values are not real corpus measurements.

12.3 Failure modes and caller experience

- An unseen station name may be converted to <UNK>, causing the system to miss the destination.
- A noisy transcript may select the wrong number or time, producing an incorrect schedule result.
- A tagger may confuse “return” as a noun or verb and therefore misinterpret the caller’s request.
- Code-mixed words may not match the English vocabulary or tagset.
- If confidence is low, the helpline should ask the caller to repeat or confirm the destination rather than silently guessing.

12.4 Human effort and cost

Manual work includes collecting and cleaning transcripts, writing annotation guidelines, hand-tagging at least 3,000 tokens, arranging independent annotation of 500 tokens, resolving disagreements, writing and testing rules, checking recogniser hypotheses and reviewing errors. Annotation quality is a continuing cost, not a one-time setup activity. Once additional rules yield little development improvement, collecting more representative labelled data is likely to be more valuable.

Task 11 conclusion Deploy only after actual held-out measurements support the choice. The system should include confidence thresholds and a confirmation path for uncertain names, dates and fares.

13. TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT

The changed scenario adds metro and intercity rail services, hundreds of new station and train names, new fare categories and increasing English–Tamil code-mixed speech. More than 15% of new-call word tokens are unseen in training. This change directly challenges the assumptions of a stable English vocabulary and a fixed domain.

13.1 A. Impact analysis

- A closed vocabulary assumption no longer holds because new station and train names arrive regularly.
- The OOV rate is no longer stable; a rate above 15% creates substantial probability and tagging problems.
- English-only word classes do not adequately describe code-mixed tokens and Tamil particles.
- Transition probabilities learned from bus-only calls may not represent metro and rail enquiry syntax.
- Proper nouns are disproportionately important: losing a station name can reverse or destroy the caller's intended route.

Perplexity is expected to increase when unseen words are mapped to <UNK> or receive poorly estimated emissions. Tagging accuracy is also expected to fall, especially on OOV tokens. The exact size of the decline must be measured on a new rail/code-mixed evaluation set; it cannot be inferred precisely from OOV rate alone.

13.2 B. Evaluate the original design

New utterance	Likely original-model failure	Why
"Metro from Perambur at six"	Perambur may be <UNK>	Unseen station name
"Chennai to Madurai bus timing enna"	Tamil "enna" may be unrecognised	English-only vocabulary and tagset
"Train to Koyambedu tomorrow varuma"	Both station and code-mixed token may fail	Combined OOV and language-switching problem

The original trigram model may assign a very low probability to the new sequences. The original HMM may classify the unknown names as generic nouns or unknown tokens rather than proper-name entities. The first component to degrade is often lexical coverage, which affects both language modelling and tagging.

13.3 C. Adaptation or redesign

The recommended choice is MODIFY rather than replace the entire system. The core n-gram and HMM architecture remains useful, but the vocabulary, smoothing, entity resources and tagset should be extended.

Component	Modification	Expected benefit
Vocabulary	Update station/train gazetteer; normalise variants; maintain <UNK>	Better coverage of important names
Language model	Add class-based n-grams for station, train and fare entities	Share statistics across entity names

Character model	Add character n-gram or subword evidence for unseen spellings	Handle phonetic and spelling variants
Smoothing	Re-estimate discounting and interpolation weights on mixed-domain data	Allocate probability to new patterns
Tagset	Add or refine CODE_MIXED / FOREIGN and transport entity labels	Represent language switching
HMM emissions	Use suffix, character, capitalisation and gazetteer features	Improve OOV tagging
Training data	Continuously add corrected metro and rail calls	Adapt to new terminology

13.4 Revised processing architecture

Layer	Revised operation
Normalisation	Preserve original script and create normalised searchable forms
Language identification	Detect English, Tamil and code-mixed spans
Entity lexicon	Match stations, trains, routes and fare categories
Language model	Interpolated n-gram + entity classes + character-level support
POS tagging	HMM with expanded tags and unknown-word features
Confidence control	Ask caller to confirm low-confidence place names or times
Monitoring	Track OOV, WER, entity recall and tag accuracy by language/domain

13.5 Refresh schedule and effort

A practical operational plan is to update the gazetteer whenever an official station or service list changes, and retrain or re-estimate the language model on a regular monthly cycle or sooner if monitoring shows a sharp OOV increase. A rolling annotation programme should prioritise new station names, code-mixed constructions and frequent recognition errors. The precise refresh interval should be adjusted using measured drift, not fixed permanently.

13.6 D. Evidence-based defence

The modified design is preferable because it preserves the efficient parts of the original system while directly addressing the new failure sources. The gazetteer protects high-value station names; class-based n-grams reduce the need to observe every name in every context; character-level evidence helps

with phonetic variants; and code-mixed tags make the annotation scheme more representative. Re-estimating interpolation weights ensures that the model reflects the new domain.

Remaining limitations include long-distance dependencies, semantic understanding, sarcasm, severe speech-recognition errors and unseen expressions that do not resemble known words. An n-gram model cannot fully represent arbitrary long-range context, even with more data, because its conditioning history is bounded by the chosen order.

Task 12 conclusion Modify the existing design with an open-vocabulary strategy, transport gazetteer, class/character support and code-mixed tagging. Monitor OOV rate, WER and tagging accuracy continuously.

14. CONCLUSION

This assignment has described a complete language-engineering workflow for a public transport enquiry helpline. N-gram language models provide a practical method for rescoring competing speech-recognition hypotheses, while POS tagging supports the extraction of entities and grammatical relationships. Unsmoothed models expose the zero-probability problem; smoothing, discounting, backoff and interpolation address sparsity. Entropy and perplexity evaluate language-model uncertainty, while WER measures the effect on transcript quality.

For POS tagging, rule-based systems are transparent but require careful rule ordering and maintenance. HMM tagging uses transition and emission probabilities with Viterbi decoding to select a globally likely sequence. Brill tagging learns readable correction rules but requires a stopping criterion to avoid overfitting. The Day-4 challenge shows that vocabulary growth and code-mixing require an adaptive design rather than a fixed English-only vocabulary.

14.1 Final recommendation

Use a smoothed interpolated n-gram model for rescoring and an HMM tagger supported by a transport gazetteer, suffix/character evidence and a code-mixed tag category. This is a provisional engineering recommendation. Final deployment must be justified with actual held-out corpus measurements, reproducible experiments and an honest effort/cost report.

14.2 Final submission checklist

- Corpus contains at least 20,000 utterances and 150,000 tokens.
- 80/10/10 split is utterance-disjoint and generated with a recorded fixed seed.
- At least 200 sets of five ASR hypotheses and reference transcripts are evaluated.
- Tagset contains 20–30 tags with a complete Penn Treebank mapping.
- At least 3,000 tokens are hand-tagged; 500 are independently double-tagged.
- Raw agreement and Cohen’s kappa are calculated from the actual annotations.
- All perplexity and accuracy tables are based on the same held-out split.
- Every illustrative value in this draft is replaced by actual experimental output.
- Code, plots, error analysis and manual effort/cost are included in the final package.

REFERENCES

1. Jurafsky, D. and Martin, J. H. Speech and Language Processing. Draft 3rd edition. Chapters on n-gram language models and part-of-speech tagging.

2. Church, K. W. and Gale, W. A. (1991). A comparison of the enhanced Good-Turing and deleted estimation methods for estimating probabilities of English bigrams.

3. Brill, E. (1995). Transformation-Based Error-Driven Learning and Natural Language Processing: A Case Study in Part-of-Speech Tagging.

4. Baum, L. E. and colleagues. Foundational work on probabilistic functions of finite-state Markov chains and HMM estimation.

5. NLTK Project. Natural Language Toolkit documentation. https://www.nltk.org/

6. Penn Treebank tagset documentation and reference descriptions.

Note: The assignment question paper is the primary source for the task requirements, constraints and mark distribution. This solution follows its 12-task structure.

APPENDIX A – EXPERIMENT RECORD SHEET

Complete this record when the real corpus and implementation are available. Keep the test set untouched until the final evaluation.

Experiment metadata	Value to enter
Corpus name / source	_____
Collection period	_____
Number of utterances	_____
Number of lexical tokens	_____
Random seed	_____
Train / development / test counts	_____
Tokeniser version	_____
Python / NLTK versions	_____
Machine specification	_____

Metric	Actual result	Notes
Vocabulary size at threshold 1		
Vocabulary size at threshold 2		
Vocabulary size at threshold 3		
Vocabulary size at threshold 5		
Development OOV rate		
Best LM perplexity		
WER before rescoring		
WER after rescoring		
Rule tagger accuracy		
HMM tagger accuracy		
Brill tagger accuracy		
Unknown-token tagging accuracy		
Inter-annotator raw agreement		
Cohen’s kappa		

APPENDIX B – MARK DISTRIBUTION FROM THE QUESTION PAPER

Task	Topic	Marks
1	Counting words and building vocabulary	3
2	Unsmoothed n-gram models	4
3	Smoothing and discounting	5
4	Backoff and deleted interpolation	4

5	Entropy, cross-entropy and perplexity	4
6	English word classes and tagset design	4
7	Rule-based POS tagging	4
8	Stochastic HMM tagging and Viterbi	5
9	Transformation-based (Brill) tagging	4
10	Implement and test the models	3
11	Compare, recommend and defend the design	2
Subtotal	Tasks 1–11	42
12	Day-4 design challenge	8
TOTAL	Assignment total	50

The assignment brief identifies the work as an individual, one-week assignment and expects measured comparisons. All numerical examples in this document are illustrative until replaced with real results.

APPENDIX C – QUICK GLOSSARY OF IMPORTANT TERMS

Term	Meaning
Acoustic model	Estimates how likely an audio signal is for a sequence of speech sounds.
Language model	Assigns probabilities or scores to word sequences.
Corpus	A collection of text or transcribed speech used for language analysis.
Tokenisation	The process of dividing text into tokens.
Vocabulary	The set of distinct tokens recognised by a model.
OOV	Out-of-vocabulary; a token not represented in the training vocabulary.
N-gram	A contiguous sequence of n items, usually words.
MLE	Maximum-likelihood estimation using relative frequency counts.
Smoothing	Adjusting estimates so unseen events can receive probability.
Discounting	Reducing some observed counts to reserve probability mass for other events.
Backoff	Using a shorter context when a longer n-gram is unavailable.
Interpolation	Combining estimates from multiple n-gram orders.

Entropy	The average uncertainty in a probability distribution.
Cross-entropy	The average coding cost when a model predicts data from another distribution.
Perplexity	The exponentiated per-token cross-entropy; lower is better under fair comparison.
POS tag	A grammatical category assigned to a word in context.
Tagset	The inventory of POS labels used for annotation.
HMM	A probabilistic sequence model with hidden states and observed emissions.
Viterbi algorithm	Dynamic programming method for finding the most likely hidden-state sequence.
Emission probability	Probability of observing a word given a POS tag.
Transition probability	Probability of a tag given the preceding tag.
Brill tagging	A transformation-based method that learns ordered correction rules.
Gazetteer	A curated list of names such as stations, towns, trains and routes.
WER	Word error rate, calculated from substitutions, deletions and insertions.

These terms summarise the central concepts used throughout the assignment and support consistent terminology during presentation or viva.